Assignment Guideline: Parcel Delivery & Courier Management System

Objective

This assignment tests your knowledge of enterprise application development by designing and implementing a fully functional Parcel Delivery & Courier Management System. The application should let customers and merchants send and track parcels, let dispatchers assign deliveries, and let couriers update delivery progress in real time. It should also integrate with external systems, as enterprise applications rarely operate in isolation. You will design, implement, secure, containerize, test, and deploy a modern enterprise application.

Weightage: 20 marks
Group Size: 10 members

Scenario

SwiftDrop is a growing courier company operating island-wide. It currently manages bookings through phone calls and spreadsheets, and customers have no visibility into where their parcels are. SwiftDrop also wants partner online stores (merchants) to create delivery orders automatically from their own e-commerce platforms. Your team has been hired to build the new platform.

Assignment Overview

1. Customer Functionality

- Secure signup and login, including "Login with Google" (OAuth 2.0).
- Create a delivery request (pickup address, drop-off address, parcel size/weight).
- View an estimated delivery fee before confirming.
- Track parcel status in real time (Created → Picked Up → In Transit → Out for Delivery → Delivered / Failed).
- View delivery history and download a delivery receipt.
- Mobile-friendly tracking page accessible via a public tracking number.

2. Merchant (Business) Functionality

- Merchant registration with an admin approval step.
- Create delivery orders via a secured API using API keys or OAuth 2.0 client credentials.
- Receive status updates through webhooks when a parcel status changes.
- Dashboard showing all orders and their delivery states.

3. Courier Functionality

- Login and authentication.
- View assigned deliveries for the day.
- Update parcel status at each stage, with a delivery note or proof (e.g., recipient name or photo upload).
- Mark failed deliveries with a reason.

4. Dispatcher/Admin Functionality

- View all unassigned parcels and assign them to couriers.
- Monitor courier workloads (number of active deliveries per courier).
- Manage users, merchants, and delivery zones/pricing.
- View basic reports (deliveries per day, success/failure rate).

5. System Requirements

- Real-time updates using WebSockets, server-sent events, or polling.
- Role-based access control for Customer, Merchant, Courier, and Dispatcher/Admin.
- JWT-based authentication for the application, plus OAuth 2.0 for social login or merchant API access.
- At least one asynchronous integration pattern (e.g., publish/subscribe via a message broker such as RabbitMQ or Kafka) for status-change events, which trigger notifications and merchant webhooks.
- A notification service (email or a mock SMS service) that consumes status events.
- Containerized deployment (frontend + backend services + database + message broker).
- Secure handling of personal data (addresses, phone numbers): input validation, hashed passwords, no sensitive data in error messages or logs.

Requirements

1. Design Documentation (20 marks)

- Use case diagrams for customer, merchant, courier, and dispatcher interactions.
- ER diagram covering users, roles, merchants, parcels, status history, couriers, zones, and pricing.
- Architecture proposal (e.g., layered, microservices, or modular monolith with a BFF) with justification, clearly showing the presentation, business logic, and data layers.
- Integration design: identify which integration patterns you used (request/reply, publish/subscribe, broker, etc.) and why.
- Sequence/activity diagrams for key flows: creating a delivery, merchant API order creation, courier status update triggering notifications, and the OAuth 2.0 login flow.
- A short note on the design patterns and principles applied (e.g., SOLID, DRY, Repository, Strategy for pricing, Observer for events).

2. Implementation (50 marks)

- **Frontend:** Responsive UI using React, Angular, or Vue.
- **Backend:** Core logic using a suitable framework (Spring Boot, Node.js/Express, Django, etc.).
- **APIs:** Secure REST or GraphQL APIs with proper versioning, validation, and centralized error handling.
- **Integration:** Message broker for status events, a notification consumer, and merchant webhooks with retry on failure.
- **Real-time updates:** Live parcel tracking via WebSockets/SSE.
- **Code quality:** Evidence of coding best practices: meaningful exception handling, no duplicated logic, configuration via environment variables (no hard-coded secrets).
- **Containerization:**
    - Dockerize all services.
    - Use docker-compose for local orchestration.

3. Testing (15 marks)

- Unit and integration tests for backend services, including at least one test for the event/integration flow.
- Tests for authorization rules (e.g., a courier cannot access another courier's deliveries).
- Measurable code coverage, with test results included in the submission.

4. Presentation/Demo (15 marks)

- Live demo of core features:
    - Customer login (including Google login), delivery creation, and real-time tracking.
    - Merchant creating an order via the API and receiving a webhook.
    - Dispatcher assigning a parcel; courier updating status.
    - Notification triggered by a status change.
- Walkthrough of architecture, diagrams, integration choices, and security decisions.
- Each member should be able to explain the part they contributed.

Bonus Marks

- **Kubernetes Deployment (+5 marks):** Deploy the full system (frontend, backend services, database, and broker) on Kubernetes using ConfigMaps, Secrets, Service definitions, and Ingress. Helm charts are a plus.
- **AI Delivery Assistant (+5 marks):** Integrate a Generative AI chatbot that can answer queries such as "Where is my parcel?" or "How much to send 2 kg to Kandy?" by calling your backend APIs. Exposing these capabilities as MCP tools is encouraged.

Submission Requirements

- Source code repository link (with a README containing setup instructions).
- Design documentation (PDF).
- Test reports and coverage summary.

- A contribution sheet listing each member's responsibilities.